UNIVERSIDADE DE MOGI DAS CRUZES
SISTEMAS DE INFORMAÇÃO

GUILHERME RAMIRES

THINDESK: DASHBOARD ADMINISTRATIVO DE CHAMADOS TÉCNICOS

MOGI DAS CRUZES
2026

GUILHERME RAMIRES

THINDESK: DASHBOARD ADMINISTRATIVO DE CHAMADOS TÉCNICOS

Trabalho apresentado à disciplina de Projeto e Desenvolvimento de Software Corporativo do curso de Sistemas de Informação da Universidade de Mogi das Cruzes, como requisito parcial para avaliação.

Professor(a): [nome do(a) professor(a)]

MOGI DAS CRUZES
2026

RESUMO

Este trabalho apresenta o desenvolvimento de um dashboard administrativo integrado ao Thindesk, sistema de chamados técnicos de uma rede de pizzarias. O objetivo é permitir que o responsável pelos chamados acompanhe o andamento do atendimento, visualize a situação dos chamados e identifique os problemas mais frequentes, apoiando a tomada de decisão. A solução foi construída em duas partes: uma API REST em Node.js com Express, ligada a um banco de dados relacional SQLite, e uma interface web em React com Bootstrap e Chart.js que consome essa API. Foram levantados os requisitos funcionais e não funcionais, definidas as regras de negócio, modelado o banco de dados e documentados os endpoints com seus parâmetros, exemplos e erros. A qualidade foi verificada por 14 testes automatizados e 20 testes manuais, todos com o resultado esperado. O código foi versionado no GitHub seguindo o modelo gitflow.

Palavras-chave: dashboard; API REST; chamados técnicos; React; Node.js.

LISTA DE ILUSTRAÇÕES

Quadro 1 – Requisitos funcionais.....6
Quadro 2 – Requisitos não funcionais.....6
Quadro 3 – Tecnologias utilizadas.....8
Quadro 4 – Tabelas do banco de dados.....8
Quadro 5 – Códigos de resposta da API.....10
Quadro 6 – Parâmetros de filtro.....10
Figura 1 – Tela principal do dashboard.....15
Figura 2 – Dashboard com filtros aplicados.....16
Figura 3 – Detalhe de um chamado aberto.....16
Quadro 7 – Testes automatizados.....18
Quadro 8 – Resumo dos testes manuais.....18
Figura 4 – Regra de técnico obrigatório na interface.....19
Figura 5 – Chamado resolvido com a solução registrada.....19
Quadro 9 – Branches do repositório.....20

SUMÁRIO

1 INTRODUÇÃO.....5
1.1 Objetivo geral.....5
1.2 Objetivos específicos.....5
1.3 Justificativa.....5
2 ANÁLISE DE REQUISITOS.....6
2.1 Regras de negócio.....6
3 PROJETO DO SISTEMA.....8
3.1 Arquitetura.....8
3.2 Tecnologias.....8
3.3 Modelagem de dados.....8
4 DOCUMENTAÇÃO DA API.....10
4.1 Filtros.....10
4.2 Endpoints.....10
GET /api/indicadores.....10
GET /api/chamados-por-periodo.....11
GET /api/situacao.....11
GET /api/tipos.....12
GET /api/filtros.....12
GET /api/chamados.....12
GET /api/chamados/:id.....13
PATCH /api/chamados/:id.....13
POST /api/chamados/:id/consideracoes.....14
5 DASHBOARD ADMINISTRATIVO.....15
6 TESTES E EVIDÊNCIAS.....18
6.1 Testes automatizados.....18
6.2 Testes manuais.....18
6.3 Testes na interface.....19
7 CONTROLE DE VERSÃO.....20
8 CONSIDERAÇÕES FINAIS.....21
REFERÊNCIAS.....22

1 INTRODUÇÃO

Redes de alimentação dependem de sistemas e equipamentos que não podem parar no horário de atendimento. Quando o caixa trava, a impressora de pedidos da cozinha deixa de imprimir ou os pedidos do aplicativo de delivery não chegam, a unidade abre um chamado técnico e espera uma solução rápida.

O Thindesk é o sistema de chamados técnicos de uma rede de pizzarias com cinco unidades. Este trabalho desenvolve o seu **dashboard administrativo**, um painel que reúne as informações dos chamados em indicadores, gráficos e listas, permitindo que o responsável acompanhe o andamento, veja a situação e as considerações técnicas de cada chamado e identifique os problemas mais recorrentes.

1.1 Objetivo geral

Analisar os requisitos, projetar e implementar um dashboard administrativo integrado ao sistema de chamados Thindesk, consumindo dados de uma API própria ligada a um banco de dados.

1.2 Objetivos específicos

a) levantar os requisitos funcionais e não funcionais do painel;
b) modelar o banco de dados dos chamados;
c) projetar e implementar os endpoints da API;
d) desenvolver a interface do dashboard integrada à API;
e) testar a solução e registrar as evidências;
f) versionar o código seguindo o modelo gitflow.

1.3 Justificativa

Sem uma visão consolidada, o responsável pelos chamados precisa abrir chamado por chamado para saber o que está atrasado ou qual tipo de problema mais se repete. O dashboard reduz esse esforço e oferece dados para decisões como trocar equipamentos que falham com frequência ou reforçar o atendimento em determinados períodos.

2 ANÁLISE DE REQUISITOS

Os requisitos foram definidos a partir do enunciado da atividade e das necessidades do responsável pelos chamados. O Quadro 1 apresenta os requisitos funcionais.

Quadro 1 – Requisitos funcionais

Código	Requisito	Descrição
RF01	Indicadores gerais	Exibir total de chamados, em aberto, atendidos, percentual dentro do prazo, tempo médio de solução e chamados atrasados.
RF02	Atendidos por período	Exibir gráfico com chamados abertos e atendidos por dia ou por mês.
RF03	Situação dos chamados	Exibir a quantidade de chamados em cada status.
RF04	Tipos mais atendidos	Exibir o ranking dos tipos de chamado com mais atendimentos.
RF05	Filtros de consulta	Filtrar por período, status, prioridade, tipo, técnico e unidade.
RF06	Integração com a API	Obter todos os dados do painel pela API.
RF07	Atualização e tratamento de dados	Alterar o status dos chamados, registrar considerações técnicas e atualizar o painel automaticamente a cada 30 segundos.

Fonte: Elaborado pelo autor (2026).

Os requisitos não funcionais, que tratam de qualidade e restrições da solução, estão no Quadro 2.

Quadro 2 – Requisitos não funcionais

Código	Requisito
RNF01	A API deve responder em formato JSON e usar os códigos HTTP 200, 201, 400, 404, 422 e 500.
RNF02	Os dados de entrada devem ser validados e as consultas ao banco devem usar parâmetros, evitando SQL injection.
RNF03	O sistema deve funcionar apenas com o Node.js 22.13 ou superior instalado, sem servidor de banco de dados.
RNF04	A interface deve ser responsiva e funcionar em computadores e celulares.
RNF05	O código deve ser versionado no GitHub seguindo o modelo gitflow.

Fonte: Elaborado pelo autor (2026).

2.1 Regras de negócio

As regras abaixo controlam o ciclo de vida dos chamados e são aplicadas pela API:

a) o prazo de atendimento depende da prioridade: alta, 8 horas; média, 24 horas; baixa, 48 horas;

b) os status possíveis são aberto, em andamento, resolvido, fechado e cancelado;
c) um chamado aberto pode ir para em andamento ou cancelado; em andamento pode ir para resolvido ou cancelado; resolvido pode ir para fechado ou voltar para em andamento;
d) para iniciar o atendimento é obrigatório ter um técnico responsável;
e) para resolver é obrigatório descrever a solução com pelo menos 10 caracteres, e essa descrição é registrada como consideração técnica;
f) chamados cancelados não recebem novas considerações.

3 PROJETO DO SISTEMA

3.1 Arquitetura

A solução segue a arquitetura cliente-servidor. O dashboard, executado no navegador, envia requisições HTTP para a API REST, que consulta o banco de dados, calcula os indicadores com SQL e devolve os resultados em JSON. Essa separação permite que outros sistemas também consumam a API no futuro.

3.2 Tecnologias

As tecnologias foram escolhidas pela simplicidade de instalação e pelo uso comum no mercado. O Quadro 3 resume a escolha.

Quadro 3 – Tecnologias utilizadas

Camada	Tecnologia	Motivo da escolha
Banco de dados	SQLite (node:sqlite)	Banco relacional completo que funciona em um único arquivo, sem instalar servidor.
API	Node.js e Express	Mesma linguagem do front-end e criação simples de rotas REST.
Interface	React	Divide a tela em componentes e atualiza os dados de forma reativa.
Estilo	Bootstrap	Componentes prontos e layout responsivo.
Gráficos	Chart.js	Gráficos de linha e barra com poucas configurações.
Testes	Vitest e Supertest	Testes automatizados das rotas com banco em memória.

Fonte: Elaborado pelo autor (2026).

3.3 Modelagem de dados

O banco possui cinco tabelas. A tabela chamados é a principal e se relaciona com unidades, categorias (tipos de chamado) e técnicos. As considerações técnicas ficam em uma tabela própria, ligada ao chamado. O Quadro 4 descreve a estrutura.

Quadro 4 – Tabelas do banco de dados

Tabela	Campos	Descrição
unidades	id, nome	Lojas da rede que abrem chamados.
categorias	id, nome	Tipos de chamado, como PDV, impressora e delivery.
tecnicos	id, nome	Técnicos responsáveis pelo atendimento.
chamados	id, titulo, categoria_id, unidade_id, tecnico_id, prioridade, status, aberto_em, prazo_em, resolvido_em	Chamados técnicos com prioridade, situação, prazo e datas.

Tabela	Campos	Descrição
consideracoes	id, chamado_id, autor, texto, criado_em	Diagnósticos, soluções e observações registradas em cada chamado.

Fonte: Elaborado pelo autor (2026).

Na primeira execução o sistema cria o banco e o preenche com cerca de 480 chamados de exemplo dos últimos quatro meses, permitindo testar o painel com dados realistas.

4 DOCUMENTAÇÃO DA API

A API responde no endereço http://localhost:3333/api. Todas as respostas são em JSON e os erros seguem o mesmo formato, com a mensagem e, quando houver, a lista de campos inválidos:

```
{ "erro": "Filtro inválido", "campos": ["data_inicio", "status"] }
```

O Quadro 5 mostra os códigos de resposta utilizados.

Quadro 5 – Códigos de resposta da API

Código	Significado
200	Requisição atendida com sucesso.
201	Registro criado com sucesso.
400	Parâmetro ou corpo inválido, ou JSON mal formado.
404	Chamado ou rota não encontrada.
422	Operação não permitida pelas regras de negócio.
500	Erro interno do servidor.

Fonte: Elaborado pelo autor (2026).

4.1 Filtros

As rotas de indicadores, gráficos e listagem aceitam os filtros opcionais do Quadro 6, que podem ser combinados.

Quadro 6 – Parâmetros de filtro

Parâmetro	Exemplo	Descrição
data_inicio	2026-10-01	Chamados abertos a partir desta data.
data_fim	2026-10-31	Chamados abertos até esta data.
status	em_andamento	aberto, em_andamento, resolvido, fechado ou cancelado.
prioridade	alta	baixa, media ou alta.
categoria_id	1	Tipo de chamado.
tecnico_id	2	Técnico responsável.
unidade_id	3	Unidade que abriu o chamado.

Fonte: Elaborado pelo autor (2026).

4.2 Endpoints

GET /api/indicadores

Retorna os indicadores gerais (RF01).
Parâmetros: filtros do Quadro 6.
Exemplo de requisição:

```
GET /api/indicadores?data_inicio=2026-10-01&data_fim=2026-10-31
```

Exemplo de resposta:

```
{
  "total": 122,
  "em_aberto": 0,
  "atendidos": 115,
  "cancelados": 7,
  "atrasados": 0,
  "sla_percentual": 73,
  "tempo_medio_horas": 21
}
```

Possíveis erros: 400 quando algum filtro é inválido.

GET /api/chamados-por-periodo

Retorna a quantidade de chamados abertos e atendidos em cada período (RF02).

Parâmetros: filtros do Quadro 6 e agrupamento, que pode ser dia (padrão) ou mes.

Exemplo de requisição:

```
GET /api/chamados-por-periodo?agrupamento=mes&data_inicio=2026-09-01
```

Exemplo de resposta:

```
[
  { "periodo": "2026-09", "abertos": 110, "atendidos": 103 },
  { "periodo": "2026-10", "abertos": 122, "atendidos": 112 },
  { "periodo": "2026-11", "abertos": 77, "atendidos": 49 }
]
```

Possíveis erros: 400 para filtro ou agrupamento inválido.

GET /api/situacao

Retorna a quantidade de chamados em cada status (RF03).

Parâmetros: filtros do Quadro 6.

Exemplo de requisição:

```
GET /api/situacao
```

Exemplo de resposta:

```
[
  { "status": "aberto", "nome": "Aberto", "total": 11 },
  { "status": "em_andamento", "nome": "Em andamento", "total": 21 },
  { "status": "resolvido", "nome": "Resolvido", "total": 14 },
  { "status": "fechado", "nome": "Fechado", "total": 406 },
  { "status": "cancelado", "nome": "Cancelado", "total": 20 }
]
```

Possíveis erros: 400 quando algum filtro é inválido.

GET /api/tipos

Retorna os tipos de chamado ordenados pelos mais atendidos (RF04).

Parâmetros: filtros do Quadro 6.

Exemplo de requisição:

```
GET /api/tipos
```

Exemplo de resposta:

```
[
  { "tipo": "PDV / Caixa", "total": 149, "atendidos": 134 },
  { "tipo": "Impressora de pedidos", "total": 99, "atendidos": 87 }
]
```

Possíveis erros: 400 quando algum filtro é inválido.

GET /api/filtros

Retorna as listas usadas nos campos de filtro do painel (RF05): categorias, técnicos, unidades e status.

Exemplo de requisição:

```
GET /api/filtros
```

Exemplo de resposta:

```
{
  "categorias": [ { "id": 6, "nome": "Acesso e senhas" } ],
  "tecnicos": [ { "id": 1, "nome": "Ana Souza" } ],
  "unidades": [ { "id": 2, "nome": "Braz Cubas" } ],
  "status": [ { "id": "aberto", "nome": "Aberto" } ]
}
```

Possíveis erros: 500 em caso de falha interna.

GET /api/chamados

Lista os chamados com paginação de 10 itens.

Parâmetros: filtros do Quadro 6, pagina (padrão 1) e busca (texto procurado no título).

Exemplo de requisição:

```
GET /api/chamados?status=em_andamento
```

Exemplo de resposta:

```
{
  "dados": [
    {
      "id": 471,
      "titulo": "Maquininha não conecta",
      "prioridade": "media",
      "status": "em_andamento",
      "aberto_em": "2026-11-12 17:14:00",
      "prazo_em": "2026-11-13 17:14:00",
      "categoria": "PDV / Caixa",
      "unidade": "Tatuapé",
      "tecnico": "Carla Mendes"
    }
  ],
  "total": 21,
  "pagina": 1,
  "paginas": 3
}
```

Possíveis erros: 400 para filtro ou página inválida.

GET /api/chamados/:id

Retorna um chamado com as suas considerações técnicas.

Parâmetros: id do chamado na URL.

Exemplo de requisição:

```
GET /api/chamados/5
```

Exemplo de resposta:

```
{
  "id": 5,
  "titulo": "Maquininha não conecta",
  "status": "fechado",
  "tecnico": "Diego Rocha",
  "consideracoes": [
    { "id": 5, "autor": "Diego Rocha", "texto": "Peça substituída." }
  ]
}
```

Possíveis erros: 400 quando o id não é numérico e 404 quando o chamado não existe.

PATCH /api/chamados/:id

Altera o status do chamado seguindo as regras de negócio (RF07).

Parâmetros: no corpo, status (obrigatório), tecnico_id (obrigatório para iniciar o atendimento se o chamado não tiver técnico) e solucao (obrigatório para resolver).

Exemplo de requisição:

```
PATCH /api/chamados/472
{ "status": "resolvido", "solucao": "Cabo de rede trocado e testado." }
```

Exemplo de resposta:

```
{
  "id": 472,
  "status": "resolvido",
  "resolvido_em": "2026-11-12 20:39:00",
  "tecnico": "Bruno Lima"
}
```

Possíveis erros: 400 para status inválido ou JSON mal formado; 404 quando o chamado não existe; 422 quando falta o técnico, falta a solução ou a mudança de status não é permitida.

POST /api/chamados/:id/consideracoes

Registra uma consideração técnica no chamado (RF07).

Parâmetros: no corpo, autor e texto (mínimo de 5 caracteres).

Exemplo de requisição:

```
POST /api/chamados/472/consideracoes
{ "autor": "Bruno Lima", "texto": "Unidade confirmou que está funcionando." }
```

Exemplo de resposta:

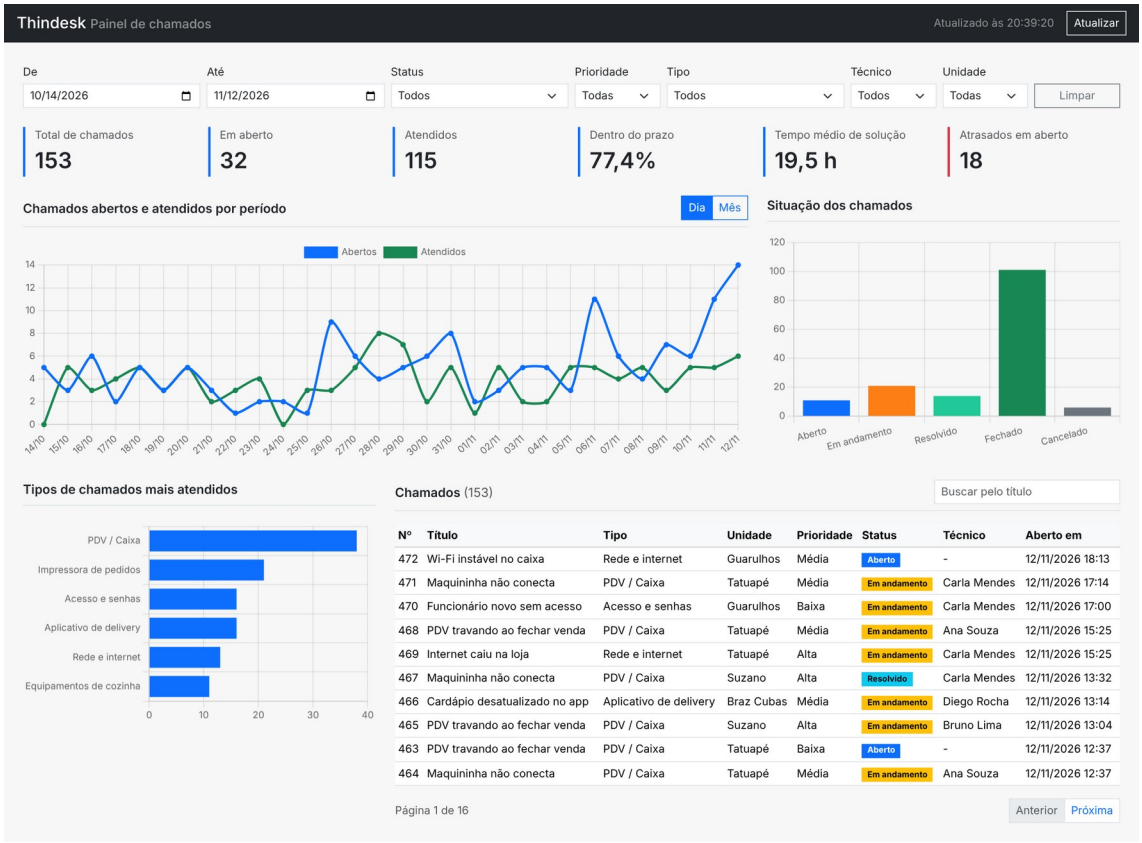
```
{
  "id": 481,
  "chamado_id": 472,
  "autor": "Bruno Lima",
  "texto": "Unidade confirmou que está funcionando.",
  "criado_em": "2026-11-12 20:39:00"
}
```

Possíveis erros: 400 sem autor ou com texto curto; 404 quando o chamado não existe; 422 quando o chamado está cancelado.

5 DASHBOARD ADMINISTRATIVO

O dashboard foi desenvolvido em React com Bootstrap. A tela principal, apresentada na Figura 1, tem no topo os filtros de consulta, em seguida os indicadores gerais e, abaixo, os gráficos de chamados por período, de situação e de tipos mais atendidos, além da lista de chamados.

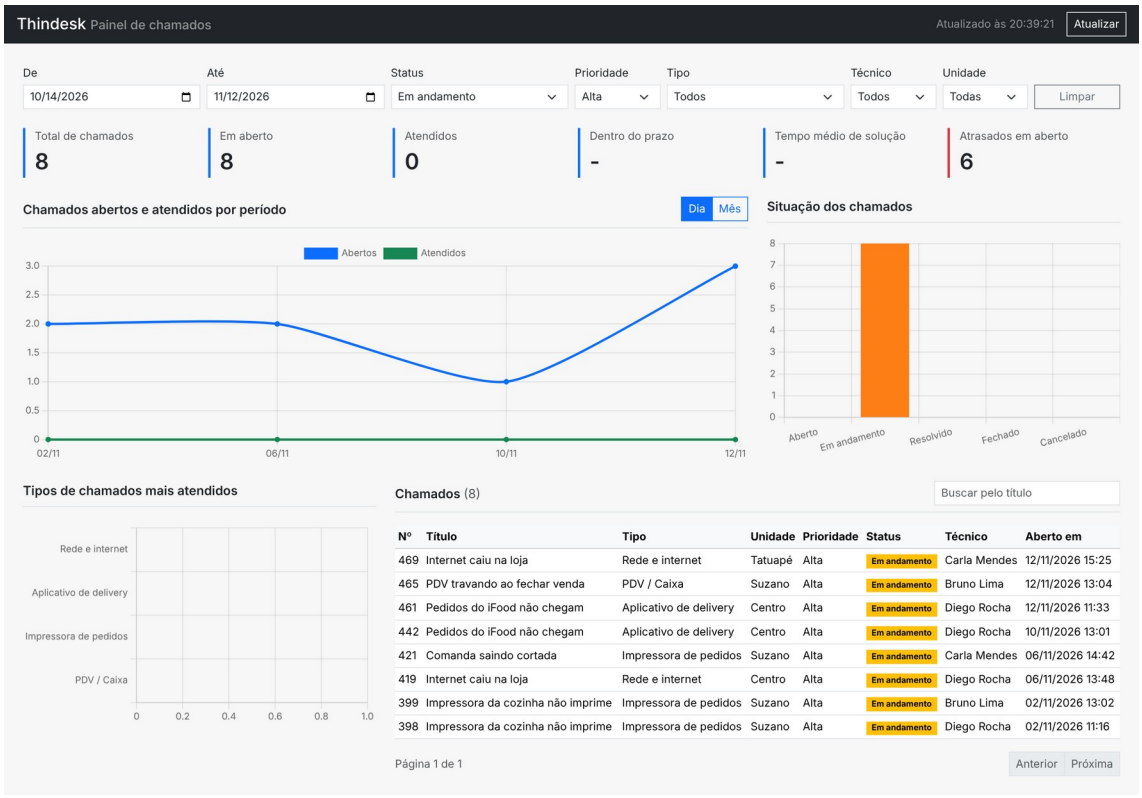
Figura 1 – Tela principal do dashboard



Fonte: Elaborado pelo autor (2026).

Os filtros se aplicam a todo o painel. A Figura 2 mostra o resultado ao filtrar os chamados em andamento com prioridade alta.

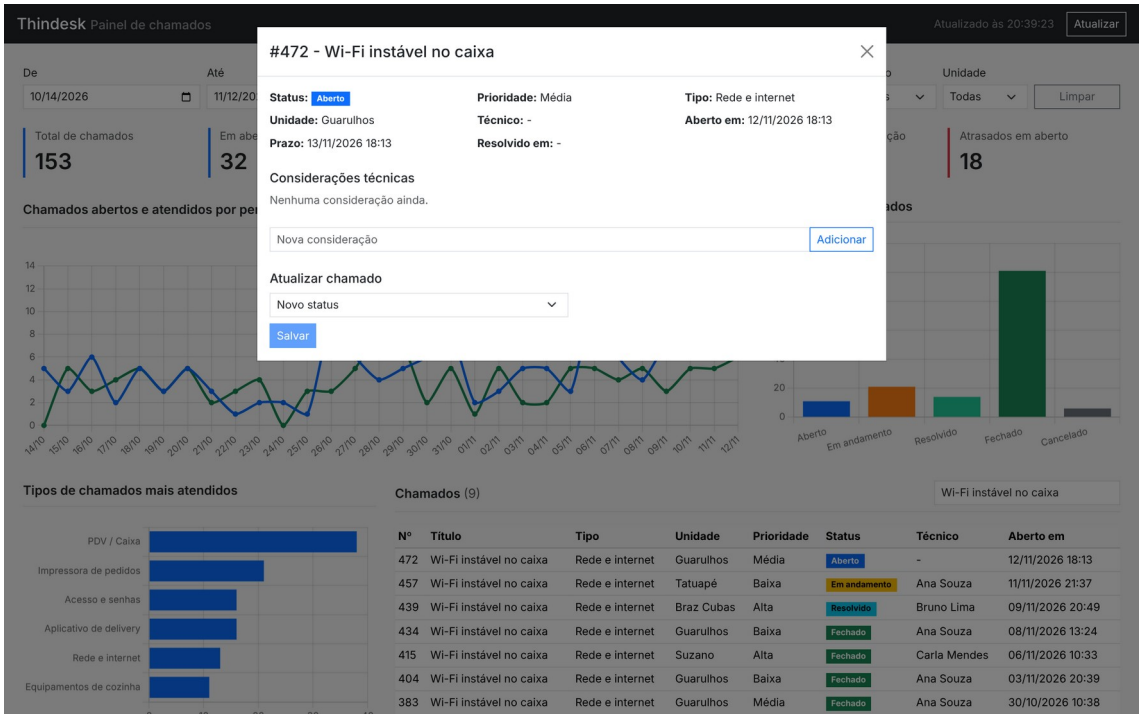
Figura 2 – Dashboard com filtros aplicados



Fonte: Elaborado pelo autor (2026).

Ao clicar em um chamado da lista, abre-se a tela de detalhe com os dados do chamado, as considerações técnicas registradas e as opções de atualização de status, como mostra a Figura 3.

Figura 3 – Detalhe de um chamado aberto



Fonte: Elaborado pelo autor (2026).

O painel recarrega os dados automaticamente a cada 30 segundos e também possui o botão Atualizar no topo da página.

6 TESTES E EVIDÊNCIAS

6.1 Testes automatizados

Foram criados 14 testes automatizados com Vitest e Supertest, que fazem requisições reais às rotas da API usando um banco de dados em memória. Todos passaram. O Quadro 7 resume o que foi verificado.

Quadro 7 – Testes automatizados

Grupo	O que é verificado	Resultado
Indicadores	Soma dos indicadores igual ao total; filtro de período; retorno zerado sem dados; erro 400 com filtro inválido.	Aprovado
Gráficos	Soma dos chamados por mês; erro 400 com agrupamento inválido; cinco status na situação; ordenação dos tipos.	Aprovado
Chamados	Paginação e filtro; detalhe com considerações; erro 404; fluxo completo de status com as regras 422; registro de consideração; JSON inválido.	Aprovado

Fonte: Elaborado pelo autor (2026).

6.2 Testes manuais

Com a API em execução, foram feitas 20 requisições cobrindo os casos de sucesso, as regras de negócio e os erros. Todas retornaram o código esperado. O registro completo, com cada requisição e resposta, está no arquivo docs/evidencias/testes-manuais.md do repositório. O Quadro 8 apresenta um resumo.

Quadro 8 – Resumo dos testes manuais

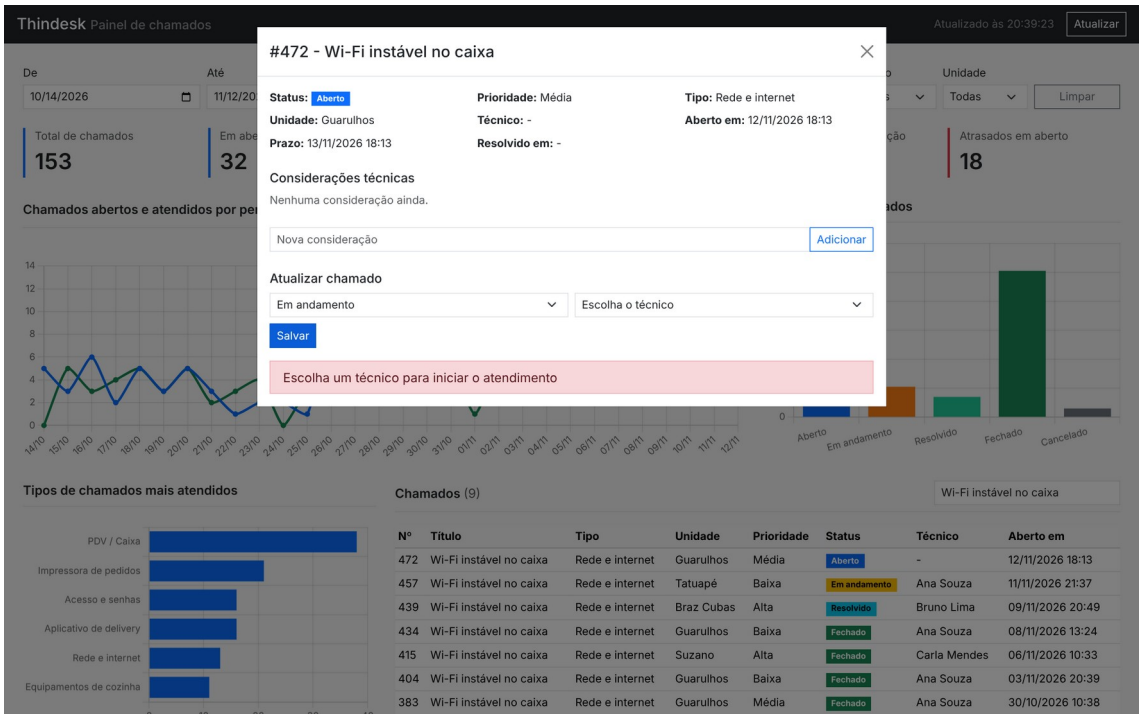
Teste	Esperado	Obtido
Indicadores de outubro	200	200
Chamados por mês	200	200
Situação dos chamados	200	200
Tipos mais atendidos	200	200
Lista de chamados em andamento	200	200
Iniciar chamado sem técnico	422	422
Resolver sem descrever a solução	422	422
Resolver com solução	200	200
Reabrir chamado fechado	422	422
Adicionar consideração técnica	201	201
Filtro com data inválida	400	400
Chamado inexistente	404	404

Fonte: Elaborado pelo autor (2026).

6.3 Testes na interface

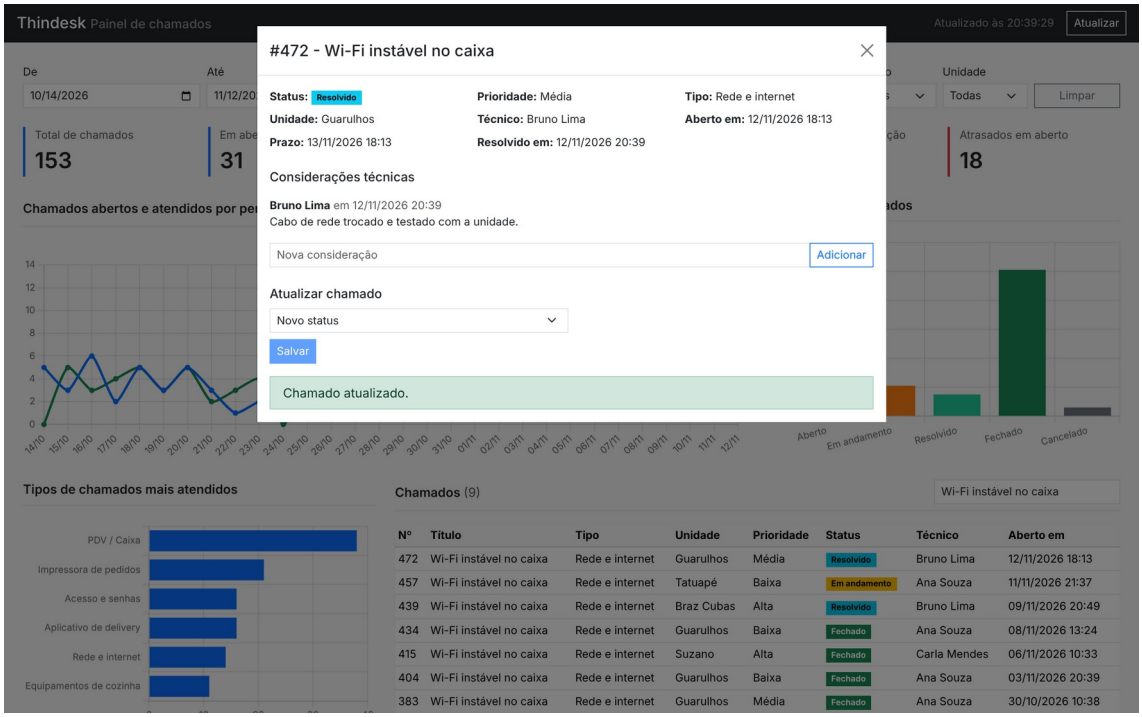
As regras de negócio também foram testadas pelo dashboard. Na Figura 4, a API recusa iniciar o atendimento sem técnico e a mensagem é exibida na tela. Na Figura 5, o chamado é resolvido e a solução aparece nas considerações técnicas.

Figura 4 – Regra de técnico obrigatório na interface



Fonte: Elaborado pelo autor (2026).

Figura 5 – Chamado resolvido com a solução registrada



Fonte: Elaborado pelo autor (2026).

7 CONTROLE DE VERSÃO

O código foi versionado no GitHub seguindo o modelo gitflow (DRIESSEN, 2010), em que cada parte do sistema é desenvolvida em uma branch própria e integrada à branch develop antes de chegar à branch principal. O Quadro 9 apresenta as branches do projeto.

Quadro 9 – Branches do repositório

Branch	**Uso**
main	Versão final do sistema.
develop	Integração das funcionalidades.
feature/banco	Criação do banco de dados e dos dados de exemplo.
feature/api	Implementação da API com os endpoints ligados ao banco e os testes.
feature/dashboard	Desenvolvimento do dashboard em React.
feature/documentacao	Documentação e evidências de testes.
release/1.0.0 e release/1.1.0	Fechamento das versões publicadas na main.

Fonte: Elaborado pelo autor (2026).

8 CONSIDERAÇÕES FINAIS

O dashboard administrativo atendeu aos sete requisitos funcionais propostos. A API fornece os indicadores, gráficos, ranking e filtros a partir do banco de dados, e a interface apresenta essas informações de forma simples, permitindo também atualizar os chamados e registrar considerações técnicas.

Os testes automatizados e manuais confirmaram o funcionamento das rotas e das regras de negócio. Como melhorias futuras, podem ser adicionados autenticação de usuários, exportação de relatórios e notificações para chamados próximos do fim do prazo.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6023**: informação e documentação: referências: elaboração. Rio de Janeiro: ABNT, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2011.

BOOTSTRAP. **Bootstrap documentation**. [S. l.], 2026. Disponível em: https://getbootstrap.com/docs/. Acesso em: 12 nov. 2026.

CHART.JS. **Chart.js documentation**. [S. l.], 2026. Disponível em: https://www.chartjs.org/docs/. Acesso em: 12 nov. 2026.

DRIESSEN, Vincent. **A successful Git branching model**. [S. l.], 2010. Disponível em: https://nvie.com/posts/a-successful-git-branching-model/. Acesso em: 12 nov. 2026.

EXPRESS. **Express: framework web para Node.js**. [S. l.], 2026. Disponível em: https://expressjs.com/pt-br/. Acesso em: 12 nov. 2026.

FIELDING, Roy Thomas. **Architectural styles and the design of network-based software architectures**. 2000. Tese (Doutorado em Ciência da Informação e Computação) – University of California, Irvine, 2000.

NODE.JS. **Node.js documentation: SQLite**. [S. l.], 2026. Disponível em: https://nodejs.org/api/sqlite.html. Acesso em: 12 nov. 2026.

REACT. **React documentation**. [S. l.], 2026. Disponível em: https://react.dev/. Acesso em: 12 nov. 2026.

VITEST. **Vitest documentation**. [S. l.], 2026. Disponível em: https://vitest.dev/. Acesso em: 12 nov. 2026.